

INF1022 - Analisadores Léxicos e Sintáticos

Expressões Regulares e Linguagens Regulares

Edward Hermann Haeusler Jefferson Santos

PUC-Rio

Departamento de Informática

Motivação

Dado um alfabeto Σ , como especificar uma linguagem $L \subseteq \Sigma^*$?

Se L é finita, uma enumeração de todas as suas cadeias/palavras¹ parece funcionar, mas funciona bem ?

$$\begin{array}{ll} \{a, aa, ab, ba, bbb, aaa\} \subseteq \{a, b, c\}^* & \{um, dois, tres\} \subseteq \{um, dois, tres\}^* \\ \{um, dois, tres\} \subseteq \{u, m, d, o, i, s, t, r, e\}^* & \{ab, a^2b, a^3b, \dots, a^{10000}b\} \subseteq \{a, b\}^* \end{array}$$

Se L é infinita, como a especificamos ?

¹strings

Alguns exemplos

Descrições informais usando Teo. Conjuntos

$$\emptyset \subseteq \Sigma^*$$

$$\Sigma^* \subseteq \Sigma^*$$

$$\{\epsilon\} \subseteq \Sigma^*$$

$$\{a, ab, aab, baa, ba\} \subseteq \{a, b\}^*$$

$$\{a^n b / n > 0\} \subseteq \{a, b\}^*$$

$$\{a^n b / n \text{ é par}\} \subseteq \{a, b\}^*$$

$$\{a^n b^m / m, n \in \mathbb{N}\} \subseteq \{a, b\}^*$$

$$\{a^n b^n / n \in \mathbb{N}\} \subseteq \{a, b\}^*$$

$$\{a^n b^j c^m / n, m \in \mathbb{N} \text{ e } 0 \leq j \leq 3\} \subseteq \{a, b, c\}^*$$

$$\{a^n b^n c^n / n \in \text{Nat}\} \subseteq \{a, b, c\}^*$$

$$\{a^p / p \text{ é primo}\} \subseteq \{a\}^*$$

Estudo do problema (I)

Só existe uma linguagem vazia, $\emptyset$

A palavra ou cadeia mais simples em qualquer alfabeto é a palavra vazia ϵ . A linguagem $\{\epsilon\}$ é uma das menores linguagens finitas.

Qualquer símbolo de um alfabeto pode ser visto como uma cadeia/palavra deste alfabeto.

Qualquer cadeia/palavra é a concatenação dos símbolos que a compoem.

Qualquer linguagem finita é a união das palavras/cadeias que peretencem a ela

Estudo do problema (II)

- (1) $\emptyset$ é linguagem sobre qualquer alfabeto Σ .
- (2) $\{\epsilon\}$ é linguagem sobre qualquer alfabeto Σ .
- (3) Para cada $\sigma \in \Sigma$, $\{\sigma\}$ é uma linguagem sobre Σ .
- (4) Se L_1 e L_2 são linguagens sobre Σ então:

$$L_1.L_2 = \{\omega_1\omega_2 : \omega_i \in L_i, i = 1, 2\}$$

é linguagem sobre Σ ,

- (5) Para toda linguagem L :

$$L = \bigcup_{\omega \in L} \{\omega\}$$

Um exemplo com conjunto finito

$$L = \{abc\ bca, ab, a, b\}$$

$$\{abc\} = \{a\}.\{b\}.\{c\} = (\{a\}.\{b\}).\{c\}$$

$$\{bca\} = \{b\}.\{c\}.\{a\} = (\{b\}.\{c\}).\{a\}$$

$$\{ab\} = \{a\}.\{b\}$$

$$L = \{abc\} \cup \{bca\} \cup \{ab\} \cup \{a\} \cup \{b\}$$

$$I = (\{abc\} \cup \{ab\} \cup \{a\}) \cup (\{bca\} \cup \{b\})$$

$$L = (\{a\}.\{b\}.\{c\}) \cup \{a\}.\{b\} \cup \{a\} \cup (\{b\}.\{c\}.\{a\}) \cup \{b\}$$

$$= \{a\}.\{b\}.\{c\} \cup \{b\} \cup \{\epsilon\} \cup \{b\}.\{c\}.\{a\} \cup \{\epsilon\}$$

Um exemplo com conjunto infinito

$$L = \{a^n / 0 \leq n\} \subseteq \{a, b\}^*$$

$$L = \{\epsilon, a, aa, aaa, aaaa, \dots\}$$

$$L = \{\epsilon\} \cup \{a\} \cup \{aa\} \cup \{aaa\} \dots$$

$$L = \{\epsilon\} \cup \{a\} . (\{\epsilon\} \cup \{a\} \cup \{aa\} \cup \{aaa\} \dots)$$

$$L = \{\epsilon\} \cup \{a\} . \underbrace{(\{\epsilon\} \cup \{a\} \cup \{aa\} \cup \{aaa\} \dots)}_L$$

$$\text{Então } L = \{\epsilon\} \cup \{a\} . L, \text{ portanto } L = \{a\}^*$$

Expressão Regular

Definição

- “ $\emptyset$ ”, “ ϵ ” e “ a ”, para todo $a \in \Sigma$ são expressões regulares (E.R.).
- Se “ e_1 ” e “ e_2 ” são expressões regulares, então “ $e_1 + e_2$ ”, “ $e_1 \cdot e_2$ ” e “ e_1^* ” são E.R.s

Expressão Regular

Definição

- “ $\emptyset$ ”, “ ϵ ” e “ a ”, para todo $a \in \Sigma$ são expressões regulares (E.R.).
- Se “ e_1 ” e “ e_2 ” são expressões regulares, então “ $e_1 + e_2$ ”, “ $e_1.e_2$ ” e “ e_1^* ” são E.R.s

Linguagem de uma E.R. , $\mathcal{L}(e)$

Definição

- $\mathcal{L}(\epsilon) = \{\epsilon\}$, $\mathcal{L}(\emptyset) = \emptyset$ e $\mathcal{L}(a) = \{a\}$.
- $\mathcal{L}(e_1.e_2) = \mathcal{L}(e_1).\mathcal{L}(e_2)$,
- $\mathcal{L}(e_1 + e_2) = \mathcal{L}(e_1) \cup \mathcal{L}(e_2)$
- $\mathcal{L}(e_1^*) = \mathcal{L}(e_1)^*$
- $\mathcal{L}(e_1^+) = \mathcal{L}(e_1)^* - \{\epsilon\}$

Algumas igualdades entre expressões regulares

$\emptyset a = a \emptyset = \emptyset$	$\varepsilon a = a \varepsilon = a$
$\emptyset^* = \varepsilon$	$\varepsilon^* = \varepsilon$
$a \cup \emptyset = a$	$a^* = (a^*)^*$
$a(b \cup c) = ab \cup ac$	$(b \cup c)a = ba \cup ca$
$(ab)^*a = a(ba)^*$	$(a \cup b)^* = (a^* \cup b)^* = a^*(a \cup b)^* = (a \cup ba^*)^* = (a^*b^*)^* = a^*(ba^*)^*$

Algumas ERs e suas respectivas linguagens

As ERs que descrevem, especificam, alguns conjuntos

$\emptyset \subseteq \Sigma^*$ é expressa por $\emptyset$

$\Sigma^* \subseteq \Sigma^*$ é expressa por $(\sigma_1 + \dots + \sigma_n)^*$

$\{\epsilon\} \subseteq \Sigma^*$ é expressa por ϵ

$\{a, ab, aab, baa, ba\} \subseteq \{a, b\}^*$ é exp. por $(a + a.b + a.a.b + b.a.a + b.a)$

$\{a^n b / n > 0\} \subseteq \{a, b\}^*$ é expressa por $a.a^*.b$

$\{a^n b / n \text{ é par}\} \subseteq \{a, b\}^*$ é expressa por $(aa)^*.b$

Algumas ERs e suas respectivas linguagens

As ERs que descrevem, especificam, alguns conjuntos

$\{a^n b^m / m, n \in \mathbb{N}\} \subseteq \{a, b\}^*$ é expressa por $a^* b^*$

$\{a^n b^n / n \in \mathbb{N}\} \subseteq \{a, b\}^*$ não possui ER que a descreva

$\{a^n b^j c^m / n, m \in \mathbb{N} \text{ e } 0 \leq j \leq 3\} \subseteq \{a, b, c\}^*$ é expressa por $a^*(\epsilon + b + b.b + b.b.b)c^*$

$\{a^n b^n c^n / n \in \mathbb{N}\} \subseteq \{a, b, c\}^*$ não possui ER que a descreva

$\{\omega / \omega \text{ inicia com } a \text{ e termina com } c\} \subseteq \{a, b, c\}^*$ é exp. por $a.(a + b + c)^*.c$

Teorema de Kleene (I)

Definition

A classe das linguagens regulares sobre Σ é a menor classe de conjunto de linguagens sobre Σ , que contêm os unitários de Σ , o conjunto $\{\epsilon\}$, o conjunto $\emptyset$, e, é fechada para união binária ($\cup$) de linguagens, concatenação ($\cdot$) de linguagens e o fecho de Kleene ($\star$).

Teorema

Uma linguagem $L \subseteq \Sigma^$ é regular, se e somente se, existe uma expressão regular E , sobre Σ , tal que $\mathcal{L}(E) = L$.*

Expressões Regulares como Ferramenta para Pattern-Matching

REGULAR EXPRESSIONS

K

[< PREV](#)

RANDOM

NEXT >

➤

WHENEVER I LEARN A
NEW SKILL I CONCOCT
ELABORATE FANTASY
SCENARIOS WHERE IT
LETS ME SAVE THE DAY.

OH NO! THE KILLER
MUST HAVE FOLLOWED
HER ON VACATION!

BUT TO FIND THEM WE'D HAVE TO SEARCH THROUGH 200 MB OF EMAILS LOOKING FOR SOMETHING FORMATTED LIKE AN ADDRESS!

IT'S HOPELESS!

EVERYBODY STAND BACK.

I KNOW REGULAR EXPRESSIONS.

- Literal Characters
- Character Classes
- Shorthand Character Classes / 'dot'
- Non-Printable Characters / Anchors
- Quantifiers
- Modifiers

Are simple characters the way you expect them.

e.g. `/a/` -> will match 'a' in a text

Pattern-Matching e o padrão POSIX

`\t` -> tab character (ASCII 0x09)

`\r` -> carriage return (0x0D)

`\n` -> line feed (0x0A)

`(\a (bell, 0x07), \e (escape, 0x1B), \f (form feed, 0x0C), \v (vertical tab, 0x0B) )`

`\xFF` -> hexadecimal index in the char. set

e.g. `\xA9` -> copyright symbol in the Latin-1

`\uFFFF` -> Unicode character

e.g. `\u20AC` -> the euro currency sign

`^` begin of the string

`$` end of the string

`\b` -> word boundary - `\B` -> not word boundary

- Groups of character
[abcdefgh0123]
- Range of character
[a-h0-3]
- Inverse Range of Character
[^i-z4-9]

e.g. `/[gG]uide[IL]ine/` -> guideline or guideLine

```
\d -> [0-9]  
\w -> [a-zA-Z]  
\s -> whitespace + [\t\r\n]
```

negative:

```
\D -> [^0-9]  
\W -> [^a-zA-Z]  
\S
```

```
'.' -> any character!!
```